



BANNARI AMMAN INSTITUTE OF TECHNOLOGY

An Autonomous Institution Affiliated to Anna University Chennai - Approved by AICTE - Accredited by NAAC with "A+" Grade

SATHYAMANGALAM - 638 401 ERODE DISTRICT TAMIL NADU INDIA

Ph: 04295-226000 / 221289 Fax: 04295-226666 E-mail: stayahead@bitsathy.ac.in Web: www.bitsathy.ac.in

22AI503/22AM503 MACHINE LEARNING

UNIT III – CLASSIFICATION

Support Vector Machines Kernel Space and Making Kernels

1. Introduction

1.1 Definition

The kernel trick extends linear SVM to handle non-linearly separable data by implicitly mapping the input feature space into a higher-dimensional (sometimes infinite-dimensional) space where a linear separator may exist, without ever explicitly computing the transformed coordinates. A kernel function $K(x,y)$ computes this implicit dot product directly from the original input vectors.

Definition:

Kernel Function: $K(x,y) = \phi(x) \cdot \phi(y)$, a function computing the dot product in some (possibly implicit) feature space, given only the original inputs x and y .

Kernel Trick: replacing every dot product $(x_i \cdot x_j)$ in the SVM dual formulation with $K(x_i, x_j)$, allowing non-linear decision boundaries without explicit feature transformation.

Feature Space: the (often much higher-dimensional) space that $\phi(x)$ implicitly maps into, where linear separation becomes possible.

1.2 Importance in Machine Learning

The kernel trick is one of the most elegant and impactful ideas in classical machine learning: it allows linear algorithms (which are well-understood, convex, and efficiently solvable) to learn highly non-linear decision boundaries simply by changing the similarity measure, without changing the underlying optimisation algorithm at all. This same trick extends beyond SVM to kernel PCA, kernel ridge regression, and Gaussian processes.

1.3 Role in the Classification Pipeline

Kernel SVM is applied when linear SVM (previous topic) underfits — i.e., when the classes are not well-separated by a straight hyperplane in the original feature space. Kernel selection and hyperparameter tuning (gamma, degree) become central design decisions in this stage of the pipeline.

2. Learning Outcomes

Understand

- The kernel trick: implicit feature mapping without explicit computation of $\phi(x)$
- The mathematical requirements (Mercer's condition) for a function to be a valid kernel
- Common kernel functions: linear, polynomial, RBF, sigmoid, and their use cases

Apply

- Implement kernel SVM using scikit-learn with appropriate kernel selection

- Tune kernel hyperparameters (gamma, degree, C) via grid search and cross-validation
- Construct valid composite kernels using closure properties (sum, product, scaling)

Analyze

- Diagnose overfitting/underfitting from the joint effect of gamma and C
- Evaluate when a custom/domain-specific kernel is warranted over standard kernels
- Compare computational trade-offs between different kernel choices at scale

3. Detailed Technical Concepts

3.1 Motivation: Why Kernels?

Many real-world classification problems are not linearly separable in their original feature space — for example, a circular decision boundary (inner class vs outer ring class) cannot be separated by any straight line in 2D. One solution is to explicitly transform the data into a higher-dimensional space via a feature mapping $\phi(x)$, where the classes may become linearly separable. However, explicitly computing $\phi(x)$ can be computationally prohibitive, especially when the target feature space is very high-dimensional or infinite-dimensional.

Figure 5.1 – The Kernel Trick: Mapping to Higher-Dimensional Feature Space

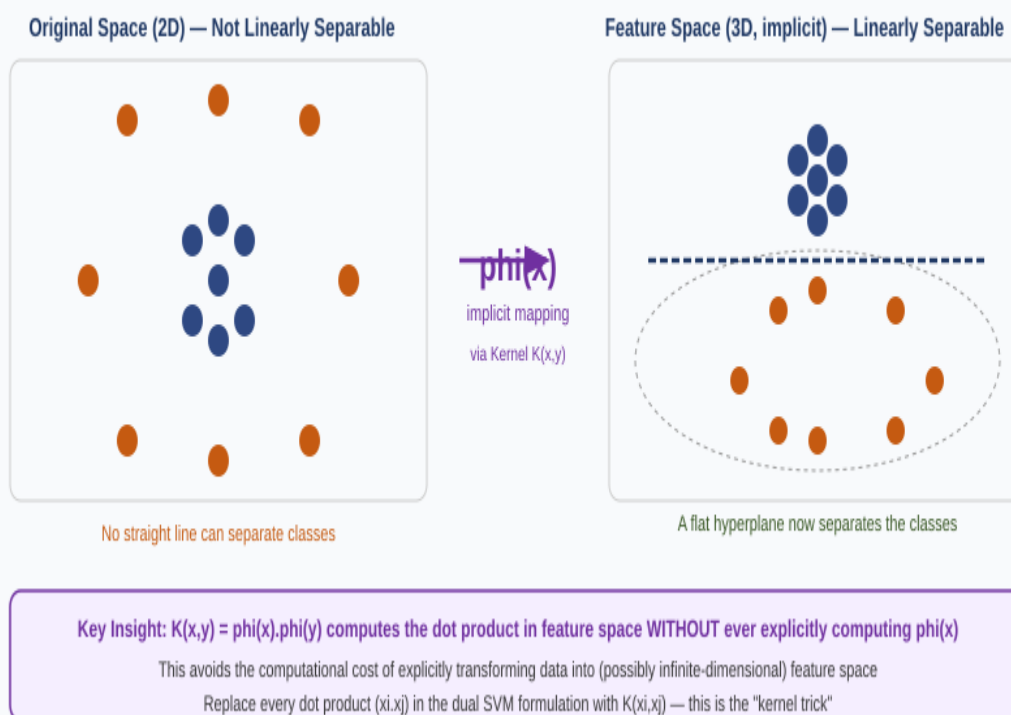


Figure 5.1 – The Kernel Trick: Mapping to Higher-Dimensional Feature Space

3.2 Working in Feature Space via the Dual Formulation

Recall from the previous topic that the SVM dual formulation depends on the training data **ONLY** through pairwise dot products $(x_i \cdot x_j)$:

maximise $\sum(a_i) - (1/2) \sum_i \sum_j a_i a_j y_i y_j (x_i \cdot x_j)$

The kernel trick replaces every occurrence of $(x_i \cdot x_j)$ with a kernel function $K(x_i, x_j) = \phi(x_i) \cdot \phi(x_j)$:

$$\text{maximise } \sum_i (a_i) - \frac{1}{2} \sum_i \sum_j a_i a_j y_i y_j K(x_i, x_j)$$

This allows the algorithm to operate as if it were working in the (possibly very high-dimensional) feature space defined by ϕ , while all computations remain in the original input space — $\phi(x)$ itself never needs to be computed. The prediction function similarly becomes: $f(x) = \sum_i (a_i y_i K(x_i, x)) + b$, again using only the kernel function, never ϕ directly.

3.3 Common Kernel Functions

3.3.1 Linear Kernel

$K(x, y) = x \cdot y$. Equivalent to standard linear SVM (no transformation). Use when data is already (near) linearly separable, or as a fast baseline before trying non-linear kernels.

3.3.2 Polynomial Kernel

$K(x, y) = (x \cdot y + c)^d$, where d is the polynomial degree and c is a constant (coef0) controlling the influence of higher-order vs lower-order terms. Captures feature interactions up to degree d . Higher d allows more complex boundaries but increases overfitting risk and computational cost.

3.3.3 RBF (Radial Basis Function / Gaussian) Kernel

$K(x, y) = \exp(-\gamma \|x - y\|^2)$, where γ controls the "width" of the similarity function. This kernel corresponds to an infinite-dimensional feature space and is the most commonly used default kernel for non-linear problems due to its flexibility and strong empirical performance across many domains.

3.3.4 Sigmoid Kernel

$K(x, y) = \tanh(\gamma x \cdot y + c)$. Inspired by neural network activation functions (behaves similarly to a two-layer perceptron). Less commonly used as a default choice since it does not always satisfy Mercer's condition (is not always a valid positive semi-definite kernel) for all parameter settings.

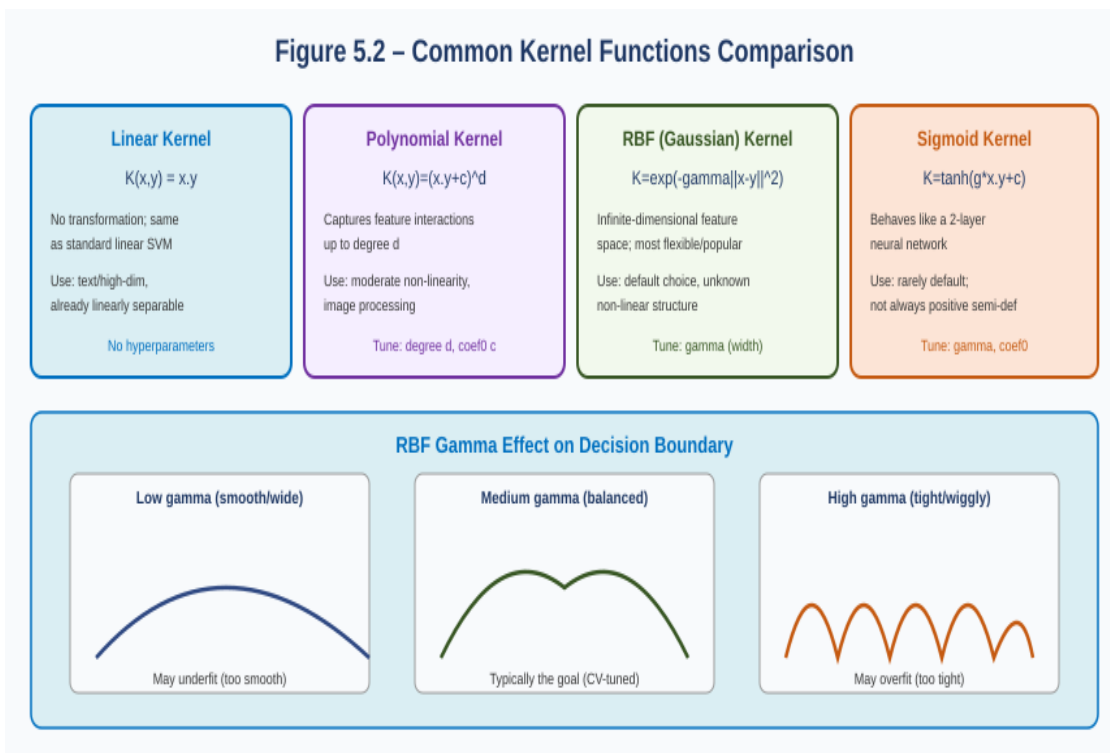


Figure 5.2 – Common Kernel Functions Comparison and the Effect of Gamma on RBF Boundaries

3.4 The Gamma Hyperparameter (RBF Kernel)

Gamma controls how far the influence of a single training example reaches:

- Low gamma: wide influence radius — smoother, simpler decision boundaries. Risk of underfitting if too low.
- High gamma: narrow influence radius — tighter, more wiggly decision boundaries closely fitting individual points. Risk of overfitting if too high.

Gamma and C jointly control model complexity: both should be tuned together via grid search with cross-validation (often on a log-scale grid, e.g., gamma in [0.001, 0.01, 0.1, 1, 10] and C in [0.1, 1, 10, 100]).

3.5 Mercer's Condition — What Makes a Valid Kernel

Not every similarity function qualifies as a valid kernel usable in SVM. Mercer's theorem states that a symmetric function $K(x,y)$ is a valid kernel (i.e., corresponds to a real dot product $\phi(x) \cdot \phi(y)$ in SOME feature space) if and only if its Gram matrix — the matrix G where $G_{ij} = K(x_i, x_j)$ for any finite set of points — is positive semi-definite (all eigenvalues are non-negative) for every possible finite set of input points.

This condition guarantees that the underlying SVM optimisation problem remains convex (has a unique, well-defined global solution) even though we never explicitly construct $\phi(x)$.

Figure 5.3 – Mercer's Condition and Making Valid Kernels

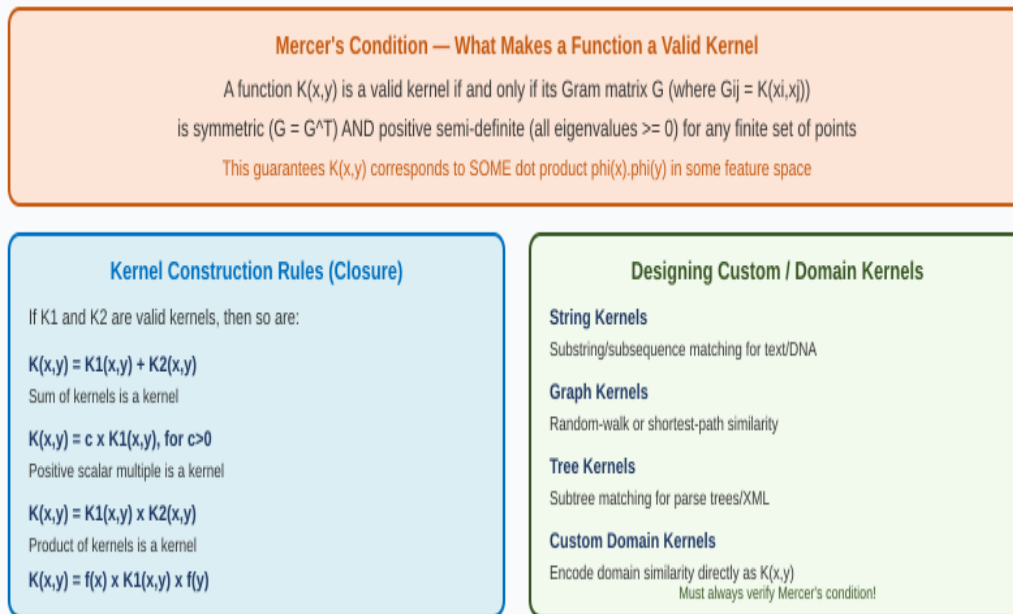


Figure 5.3 – Mercer's Condition and Rules for Constructing Valid Kernels

3.6 Making Kernels: Closure Properties

New valid kernels can be systematically constructed from existing valid kernels using closure properties — this is the practical toolkit for "making kernels" referenced in the syllabus. If $K_1(x,y)$ and $K_2(x,y)$ are both valid kernels, then the following are also guaranteed to be valid kernels:

- Sum: $K(x,y) = K1(x,y) + K2(x,y)$ — combines two notions of similarity additively.
- Scaling: $K(x,y) = c * K1(x,y)$ for any constant $c > 0$ — adjusts the influence of a kernel.
- Product: $K(x,y) = K1(x,y) * K2(x,y)$ — combines two similarity notions multiplicatively (e.g., combining a text-similarity kernel with a metadata-similarity kernel).
- Function scaling: $K(x,y) = f(x) * K1(x,y) * f(y)$ for any function f — reweights the kernel by a feature-dependent scalar.
- Polynomial transform: $K(x,y) = p(K1(x,y))$ for any polynomial p with non-negative coefficients.
- Exponential: $K(x,y) = \exp(K1(x,y))$ — also guaranteed valid.

These rules allow practitioners to engineer custom, domain-appropriate kernels by combining simpler building-block kernels, while guaranteeing the result remains mathematically valid (positive semi-definite) without needing to re-derive Mercer's condition from scratch each time.

3.7 Domain-Specific / Custom Kernels

Beyond the standard kernels, domain-specific kernels can directly encode meaningful similarity for structured or non-vector data:

- String Kernels: measure similarity between sequences (text, DNA, protein sequences) based on shared substrings or subsequences — used in bioinformatics and NLP without needing explicit fixed-length feature vectors.
- Graph Kernels: measure similarity between graph-structured data (molecules, social networks) via random walks, shortest paths, or subgraph matching.
- Tree Kernels: measure similarity between hierarchical structures (parse trees, XML documents) via shared subtree counting.
- Custom domain kernels: practitioners can encode any domain-appropriate notion of similarity as $K(x,y)$, provided Mercer's condition is verified (or empirically validated via a positive semi-definite Gram matrix check).

3.8 Choosing a Kernel

Scenario	Recommended Kernel	Rationale
High-dimensional sparse data (text, TF-IDF)	Linear	Already near-linearly separable; non-linear kernels add cost without benefit
Unknown non-linear structure, moderate dimensions	RBF	Most flexible default; strong general-purpose performance
Known polynomial-degree feature interactions	Polynomial	Explicitly models interaction terms up to chosen degree
Structured/sequential data (text, DNA, graphs)	Custom (string/graph/tree)	Captures domain-specific similarity not expressible as fixed vectors
Very large n (100,000+ samples)	Linear (LinearSVC)	Non-linear kernel training scales poorly ($O(n^2)$ to $O(n^3)$) for large n

4. Architecture / Kernel SVM Workflow

Step	Action	Method / Code	Output
1	Scale features	<code>StandardScaler().fit_transform(X)</code>	Standardised feature matrix

2	Assess linear separability	Try LinearSVC baseline first	Baseline accuracy / decision on kernel necessity
3	Select candidate kernel(s)	kernel="rbf"/"poly"/"sigmoid"/custom	Candidate kernel functions
4	Grid search C and gamma jointly	GridSearchCV over C and gamma (log-scale)	Optimal hyperparameter combination
5	Verify Mercer's condition (custom kernels)	Check Gram matrix eigenvalues ≥ 0	Validated kernel function
6	Train final model	SVC(kernel=..., C=..., gamma=...).fit(X,y)	Trained kernel SVM model
7	Validate on held-out test set	accuracy_score, classification_report	Final performance metrics

5. Industry Tools and Technologies

5.1 Commercial Tools

Tool	Vendor	Kernel SVM Features
MATLAB Statistics Toolbox	MathWorks	fitsvm with RBF, polynomial, custom kernel functions
SAS Enterprise Miner	SAS	PROC SVM with multiple kernel options
RapidMiner	RapidMiner	SVM operator (LibSVM) with full kernel parameter control
KNIME	KNIME	SVM Learner node with kernel selection widget
Weka	Univ. of Waikato	SMO classifier with PolyKernel, RBFKernel options

5.2 Open-Source Tools

- scikit-learn: SVC(kernel="rbf"/"poly"/"sigmoid"/"precomputed"), custom kernel support via callable
- LIBSVM: the foundational reference C++ implementation with RBF, polynomial, sigmoid kernels
- shogun: ML toolkit with extensive kernel library including string and graph kernels
- GPy / GPflow: Gaussian Process libraries sharing the same kernel design philosophy
- e1071 (R): svm() with kernel="radial"/"polynomial"/"sigmoid" options

6. Real-World Applications

Domain	Application	Kernel Used	Why This Kernel
Bioinformatics	Protein/DNA sequence classification	String kernel	Sequences have variable length, no natural fixed-vector form
Computer Vision (pre-deep-learning)	Image classification on extracted features	RBF / Histogram Intersection	Captures non-linear visual similarity

Chemistry	Molecular property prediction	Graph kernel	Molecules are naturally graph-structured
Handwriting Recognition	Digit/character classification	Polynomial / RBF	Captures non-linear stroke pattern interactions
NLP	Document similarity for clustering/retrieval	String / Linear (TF-IDF)	Text similarity via shared substrings or sparse vectors

7. Advantages and Limitations

Advantages	Limitations
Enables linear algorithms to learn non-linear boundaries efficiently	Computational cost scales poorly with n ($O(n^2)$ to $O(n^3)$ for kernel matrix)
No need to explicitly compute high/infinite-dimensional features	Choosing the right kernel and tuning γ /degree/ C requires careful experimentation
Closure properties allow systematic construction of custom kernels	Custom kernels require verifying Mercer's condition (non-trivial in general)
RBF kernel is a strong, flexible general-purpose default	Kernel SVM does not scale well to very large datasets (millions of samples)
Domain-specific kernels (string, graph) handle structured, non-vector data elegantly	Less interpretable than linear SVM — harder to explain individual predictions

8. Case Study

CASE STUDY

Case: Protein Classification with String Kernels (Leslie et al., 2002)

Problem: Classifying protein sequences into structural/functional families is a fundamental bioinformatics task, but proteins are variable-length sequences of amino acids — not naturally expressible as fixed-length numeric feature vectors required by standard SVM.

Dataset: SCOP (Structural Classification of Proteins) database, thousands of protein sequences across dozens of structural families.

Approach: Leslie, Eskin, and Noble introduced the "spectrum kernel" — a string kernel computing similarity between two protein sequences based on the count of shared k -length subsequences (k -mers), without ever explicitly constructing a feature vector.

Kernel Definition: $K(x,y) = \sum \text{over all possible } k\text{-mers of } (\text{count of } k\text{-mer in } x) \times (\text{count of } k\text{-mer in } y)$. This is provably a valid Mercer kernel (it is literally a dot product in the explicit, if extremely high-dimensional, space of all possible k -mer counts).

Findings: SVM with the spectrum kernel substantially outperformed previous methods (profile HMMs and simple BLAST-based nearest-neighbour approaches) on remote protein homology detection — correctly classifying proteins into families even when sequence similarity to training examples was low.

Conclusion: This work demonstrated the power of "making kernels" for structured, non-vector data — by designing $K(x,y)$ to directly encode meaningful biological similarity, SVM could be applied to a domain

where standard feature engineering was previously a major bottleneck, without ever needing to construct explicit high-dimensional feature vectors.

9. Scenario-Based Learning

SCENARIO 1

SCENARIO 1: Linear SVM Underfits a Clearly Non-Linear Problem

Background: A quality-control team uses linear SVM to classify products as defective/non-defective based on two sensor readings (vibration_amplitude, temperature_deviation), where defective products form a ring/donut pattern around a central cluster of acceptable products in feature space.

Technical Environment: Python, scikit-learn LinearSVC, StandardScaler, 8,000 product samples.

Problem Statement: Linear SVM achieves only 54% test accuracy (barely better than random for this roughly balanced binary problem) despite extensive C tuning.

Available Evidence: 2D scatter plot of the (scaled) features clearly shows a circular/ring decision boundary is needed — no straight line can separate the central "good" cluster from the surrounding "defective" ring.

Investigation: Since the problem is inherently non-linear (verified visually and via the persistently poor linear SVM accuracy across all C values), switch to a non-linear kernel.

Analysis: Apply SVC(kernel="rbf") with grid search over C and gamma. The RBF kernel's implicit infinite-dimensional feature space can represent the required circular/elliptical decision boundary.

Findings: RBF kernel SVM with tuned gamma and C achieved 96.8% test accuracy — a dramatic improvement, fully capturing the ring-shaped decision boundary that linear SVM fundamentally could not represent.

Recommended Solution: Always visualise the data (or check linear SVM performance as a diagnostic) before committing to a linear model. When linear models persistently underperform despite proper scaling and C tuning, this is a strong signal to try kernel SVM (RBF as the default first choice).

SCENARIO 2

SCENARIO 2: Overfitting from Excessively High Gamma

Background: A team applies RBF kernel SVM to a customer churn prediction problem with 15 features and 5,000 training samples, using gamma="auto" initially, then manually increasing gamma to "improve" training accuracy.

Technical Environment: Python, scikit-learn SVC(kernel="rbf"), 5,000 customer records, 80/20 train/test split.

Problem Statement: As gamma was increased from 0.01 to 1.0 to 10.0, training accuracy rose from 87% to 99.8% to 100%, but test accuracy fell from 85% to 79% to 68% — a classic overfitting pattern.

Available Evidence: At gamma=10.0, the decision boundary visualised on a 2D PCA projection shows extremely tight, isolated "islands" around individual training points rather than smooth class regions — the model has essentially memorised the training set.

Investigation: Plot training vs validation accuracy across a range of gamma values (log-scale) to identify the point of divergence (where validation accuracy starts decreasing while training accuracy keeps increasing).

Analysis: High gamma creates an overly narrow influence radius per support vector, causing the model to fit noise in the training data rather than the true underlying class structure — exactly analogous to small k in kNN.

Findings: Cross-validated grid search identified gamma=0.05 and C=1.0 as optimal: training accuracy 89%, test accuracy 87% — properly balanced and far more robust than the "high training accuracy" gamma=10 configuration.

Recommended Solution: Never select hyperparameters based on training accuracy alone. Always use cross-validation to jointly tune gamma and C, and treat a large train/test accuracy gap as a clear overfitting warning sign requiring lower gamma and/or lower C.

SCENARIO 3

SCENARIO 3: Custom Kernel for Variable-Length Time-Series Classification

Background: A predictive maintenance team needs to classify equipment sensor readings (time-series of varying length, since different equipment runs are monitored for different durations) as Normal/Anomalous.

Technical Environment: Python, scikit-learn SVC(kernel="precomputed"), tslearn library, 2,500 time-series samples of varying lengths (50 to 500 time-steps).

Problem Statement: Standard kernels (linear, RBF, polynomial) all require fixed-length feature vectors, but the variable-length time-series cannot be naturally truncated or padded without losing or distorting information.

Available Evidence: Naive approaches (truncating to shortest length, or zero-padding to longest length) degraded accuracy to 71% and 68% respectively due to information loss/distortion.

Investigation: Research domain-appropriate similarity measures for variable-length sequences — Dynamic Time Warping (DTW) is a well-established distance measure for time-series that handles variable length and temporal misalignment naturally.

Analysis: Construct a custom Gram matrix using a Global Alignment Kernel (a smoothed, provably positive-semi-definite variant of DTW-based similarity, since raw DTW distance does not satisfy Mercer's condition directly) and feed this precomputed kernel matrix into SVC(kernel="precomputed").

Findings: SVM with the Global Alignment Kernel achieved 91.3% accuracy, substantially outperforming both naive fixed-length approaches and a baseline 1-NN-DTW classifier (84.7%).

Recommended Solution: When data does not naturally fit a fixed-length vector representation (variable-length sequences, graphs, trees), research and apply domain-appropriate custom kernels rather than forcing data into an ill-suited fixed-vector format — but always verify the kernel's validity (Mercer's condition) before use, since not all natural similarity measures (like raw DTW) qualify.

10. Common Errors and Troubleshooting

Error	Cause	Impact	Fix
Using RBF kernel by default without trying linear first	Skipping the simpler baseline	Unnecessary computational cost for linearly separable data	Always benchmark LinearSVC first; only move to RBF if it underperforms
Tuning gamma/C using training accuracy	Not using a validation set or CV	Severe overfitting, poor test performance	Always tune via cross-validation, monitor train/test gap
Using a non-Mercer-valid custom similarity function	Not checking positive semi-definiteness	Optimisation may not converge or finds invalid solution	Verify Gram matrix eigenvalues, or use a known-valid kernel (e.g., Global Alignment for DTW)
Kernel SVM on very large datasets	Not considering computational complexity	Training becomes impractically slow ($O(n^2)$ - $O(n^3)$)	Use linear SVM, subsample, or approximate kernel methods (Nystrom, random features) for large n
Forgetting to scale features before kernel SVM	Habit not carried over from linear SVM	RBF/polynomial kernels distorted by feature scale differences	Always StandardScale features before any kernel SVM

11. Key Terminologies

Term	Definition
------	------------

Kernel Function $K(x,y)$	Function computing the dot product $\phi(x) \cdot \phi(y)$ in feature space without explicit ϕ
Kernel Trick	Replacing dot products with kernel function evaluations to enable non-linear learning
Feature Space	The (possibly implicit, high/infinite-dimensional) space that $\phi(x)$ maps into
Mercer's Condition	Requirement that the Gram matrix be symmetric and positive semi-definite for K to be valid
Gram Matrix	Matrix G where $G_{ij} = K(x_i, x_j)$ for a given set of training points
RBF (Radial Basis Function) Kernel	$K(x,y) = \exp(-\gamma \ x-y\ ^2)$; flexible, infinite-dimensional, common default
Gamma (RBF parameter)	Controls the width of influence of each training point in RBF kernel
Polynomial Kernel	$K(x,y) = (x \cdot y + c)^d$; captures feature interactions up to degree d
Sigmoid Kernel	$K(x,y) = \tanh(\gamma x \cdot y + c)$; neural-network-like, not always Mercer-valid
Kernel Closure Properties	Rules (sum, product, scaling) for constructing new valid kernels from existing ones
String Kernel	Kernel measuring similarity between sequences via shared substrings/subsequences
Precomputed Kernel	A custom Gram matrix supplied directly to SVM instead of computed from raw features

12. Topic Summary

UNIT III – Topic 5

SVM — KERNEL SPACE AND MAKING KERNELS — SUMMARY

- Kernel trick: replace dot products $(x_i \cdot x_j)$ in the SVM dual formulation with $K(x_i, x_j)$, enabling non-linear boundaries without explicit high-dimensional feature computation.
- Common kernels: Linear (no transform), Polynomial (degree- d interactions), RBF/Gaussian (infinite-dim, flexible default), Sigmoid (neural-net-like, less common).
- Gamma (RBF) and C jointly control model complexity: high gamma/high C risk overfitting; low gamma/low C risk underfitting. Always tune jointly via cross-validation.
- Mercer's condition: a function is a valid kernel only if its Gram matrix is symmetric and positive semi-definite for any finite point set.
- Making kernels: closure properties (sum, scaling, product, function-scaling, polynomial, exponential) allow systematic construction of new valid kernels from existing ones.
- Custom/domain kernels (string, graph, tree) extend SVM to structured, non-vector data (sequences, molecules, parse trees) where fixed-length feature vectors are unnatural.
- Kernel SVM scales poorly to very large n ($O(n^2)$ - $O(n^3)$); prefer linear SVM or approximate methods (Nystrom, random features) at scale.
- Tools: scikit-learn (SVC with kernel parameter, custom/precomputed kernels), LIBSVM, shogun, tslearn for time-series kernels.